

EK Web Development

Sikkerhed for Webudviklere

Eksamensprojekt - BoligPortalen A/S

Navn: Julie Kristina Jensen
Studienummer: juje0003
Afleveringsdato: 1. juni 2026

1. Introduktion

Formål

Jeg har udviklet en simpel webapplikation, der simulerer en boligportal. Jeg har hentet inspiration fra sider som Zillow, boligsiden.dk og danbolig. Min boligportal er beregnet til almindelige brugere, der kan bruge siden til at finde boliger til salg.

På siden har brugerne adgang til et kort, hvor de kan se omkring 40.000 boliger. De kan bruge søgefunktionen og vælge, hvor mange badeværelser og værelser boligen skal have. I søgefunktionen kan de kun søge efter bynavne, og resultaterne vises på kortet. Brugere kan selv zoome ind og ud på kortet og klikke på de små ikoner for hver bolig for at se information om den pågældende bolig.

bruger som er logget ind kan også uploade et billede til deres profil billede, gemme og fjerne igen fra sin gemte liste af boliger.

Brugere af websitet er opdelt i to roller: admin og almindelig bruger. En almindelig bruger er en, der har oprettet en profil og verificeret den via e-mail. Brugere med admin-rollen kan oprette nye boliger, administrere tilbud fra logget ind bruger om boliger

Baggrund: Beskriv den fiktive virksomhed (branche, antal medarbejdere, behov).

BoligPortalen A/S er en landsdækkende dansk ejendomsmæglervirksomhed med kontorer i alle landets kommuner. Der er ca. 700 ansatte fordelt på mere end 140 selvstændige butikker i Danmark. På selve hovedkontoret er der cirka 25 ansatte tilknyttet. Virksomheden formidler aktivt cirka 44.000 boliger til salg og er dermed en af Danmarks største aktører inden for ejendomsformidling.

Virksomhedens digitale platform er central for forretningen, kunder bruger boligportalen til at finde, gemme og forespørge på boliger, mens mæglere og administratorer opretter og administrerer boliglistinger. Da platformen håndterer persondata for tusindvis af brugere samt følsomme oplysninger om ejendomshandler, er IT-sikkerhed en kritisk prioritet.

Teknologier og kendte sårbarheder

Følgende teknologier er anvendt i udviklingen af boligportalen:

Teknologi	Version	Formål
PHP	8.2	Backend sprog
mixHTML	Custom	Templating og routing

Composer	2.x	Pakkehåndtering
PHPMailer	6.x	Email-afsendelse
MariaDB	10.6	Relationel database
HTML/CSS/JavaScript	-	Frontend
Leaflet.js	1.x	Kortvisning
Docker	-	Containerisering
Nginx	Latest	Reverse proxy
phpMyAdmin	Latest	Database GUI (kun dev)

Kendte sårbarheder og overvejelser:

PHP 8.2 er en aktiv og understøttet version med regelmæssige sikkerhedsopdateringer. En historisk PHP-sårbarhed der er relevant at kende til er type juggling ved løs sammenligning med `==` kan værdier som `0 == "abc"` evaluere til `true`, hvilket kan omgå autentificerings tjek. I mit projekt bruger jeg `===` og `hash_equals()` til alle sikkerhedskritiske sammenligninger.

PHPMailer har historisk haft kritiske sårbarheder CVE-2016-10033 tillod Remote Code Execution via manipuleret afsenderadresse. Nyere versioner har lukket denne sårbarhed. Jeg holder PHPMailer opdateret via `composer.lock` og kan verificere at der ikke er kendte sårbarheder med:

```
● PS C:\2026_1_php> composer audit
No security vulnerability advisories found.
```

MariaDB arver mange sikkerhedsegenskaber fra MySQL. Standard root-password uden begrænsninger er en klassisk fejlkonfiguration i mit udviklingsmiljø bruges et simpelt password, men i produktion ville dette erstattes med et stærkt password og en dedikeret databasebruger med minimale rettigheder i stedet for root.

Leaflet.js og **Font Awesome** indlæses fra eksterne CDN'er (unpkg.com og kit.fontawesome.com), hvilket udgør en supply chain-risiko som beskrevet i afsnit 4.11. CDN-ressourcer er eksplicit whitelisted i CSP-headeren.

mixHTML er et hjemmelavet framework uden et offentligt community der finder og rapporterer sårbarheder. Frameworket er udviklet af Santiago. Det betyder at

sikkerhedshåndtering er mit eget ansvar, der er ingen automatisk beskyttelse mod XSS, CSRF eller SQL injection som man ville få med et modent framework som Laravel.

Nginx som reverse proxy skjuler den bagvedliggende PHP-server fra internettet og tilføjer et ekstra lag mellem angriberen og applikationen. `server_tokens off` er sat for at forhindre at Nginx afslører sin version i HTTP-headers.

2. Projektbeskrivelse

Funktionelle krav:

Multilevel login (admin/user)

I mit system har jeg implementeret et multilevel login for at skelne mellem admin og almindelige brugere. I databasen har jeg oprettet to tabeller, der håndterer roller: en tabel kaldet `roles`, som indeholder to kolonner, `role_name` og `role_pk`, og en join-tabel kaldet `role_table`, der refererer til brugerens primærnøgle (`user_pk`) og rollens primærnøgle (`role_pk`).

Dette design er valgt, så `users`- og `roles`-tabellerne aldrig interagerer direkte med hinanden. I stedet refererer `join`-tabellen simpelthen til både brugeren og rollen. På denne måde undgås redundans, og systemet bliver mere fleksibelt, da roller nemt kan tilføjes eller ændres uden at skulle opdatere brugerdata.

Når jeg skal tjekke, hvilken rolle en bruger har, bruger jeg en stored procedure i databasen. Denne procedure henter `role_name` ved at udføre et join mellem `users`, `role_table` og `roles` for at finde brugerens rolle ud fra deres `user_pk`. SQL-koden ser sådan ud:

```
sql
Copy
BEGIN
    DECLARE role_name VARCHAR(50);

    SELECT roles.role_name INTO role_name
    FROM users
    JOIN role_table ON users.user_pk = role_table.user_fk
    JOIN roles ON role_table.role_fk = roles.role_pk
    WHERE users.user_pk = user_pk;

    RETURN role_name;
END
```

Denne funktion tager simpelthen et `user_pk` som input, hvilket jeg henter, når brugeren logger ind. For at sikre mig mod SQL injection sender jeg `user_pk` til databasen ved hjælp af placeholders, så inputtet aldrig ufortolket indsættes i SQL-koden.

Når `role_name` er hentet, gemmes den i brugerens session sammen med resten af brugerens data. Adgangskoden fjernes dog fra sessionen, før den gemmes, for at undgå at den ved et uheld bliver eksponeret.

Jeg har gået ud fra Princippet om Mindste Privilegium (PoLP) , som går ud på, at brugere og processer kun skal have de minimum rettigheder, der er nødvendige for at udføre deres opgaver og intet mere. Dette princip reducerer risikoen for misbrug, fejl og sikkerhedsbrud, da en bruger eller proces ikke kan gøre mere, end de er designet til.

Jeg bruger polp i mit system ved at Adskillelse af Roller og Brugere. Rolle og brugere er adskilt i separate tabeller (roles og users), og forbindelsen mellem dem håndteres via en join-tabel (role_table). Dette gør at en bruger kun får de rettigheder, der er knyttet til deres rolle (f.eks. admin eller user). Ingen bruger automatisk får adgang til funktioner, de ikke har behov for. For eksempel kan en almindelig bruger ikke slette boliger eller se andre brugere, da disse funktioner er forbeholdt admin-rollen.

Rollebaseret Adgangskontrol (RBAC)

Når en bruger logger ind, hentes deres rolle via den stored procedure, jeg har implementeret. Denne rolle gemmes i sessionen og bruges til at begrænse adgangen til funktioner i systemet. Dette gør at hvis en bruger har rollen user, får de kun adgang til at søge efter boliger og se informationer. Hvis en bruger har rollen admin, får de udvidet adgang til at oprette, slette og markere boliger som "private".

Jeg har også tænkt meget over hvordan jeg sikkert håndtere mine session. Jeg har valgt at fjerne adgangskoden fra sessionen, før den gemmes. Dette går igen under polp dat sessionen kun indeholder de data, der er nødvendige for at opretholde brugerens login-status og rolle. Følsomme data som adgangskoden aldrig gemmes unødigt, hvilket reducerer risikoen for, at de bliver kompromitteret.

I alle mine SQL queries bruger jeg Prepared Statements for at undgå sql-injections. Ved at bruge placeholders i mine SQL queries sikrer jeg mig at brugerinput aldrig kan manipulere forespørgslen. Dette er en teknisk implementering af PoLP, da databasen kun udfører de forespørgsler, den er designet til, ingen uventet eller uautoriseret adgang til data.

```
$sql = "SELECT get_user_role(:user_pk) AS role_name";

$stmt = $_db->prepare($sql);

$stmt->bindValue(":user_pk",$user['user_pk']);

$stmt->execute();

$user_role = $stmt->fetch();
```

Denne implementering sikrer, at brugerens rolle altid er korrekt hentet og gemt, og at systemet er beskyttet mod almindelige sikkerhedsrisici som SQL injection og session hijacking. Samtidig gør det det nemt at udvide systemet med flere roller i fremtiden, hvis det bliver nødvendigt.

Brugerregistrering med email-validering

For at sikre, at kun ægte brugere kan oprette en konto på boligportalen, kræves email-validering. Når en ny bruger registrerer sig, skal de angive en gyldig email-adresse. Efter registrering modtager brugeren et verifikationslink på den angivne email. Dette link indeholder brugerens email som parameter (f.eks. /verify?email=bruger@example.com). Når brugeren klikker på linket, sendes en HTTP-request til serveren med emailen som parameter. Serveren udfører følgende trin for at validere brugeren:

1. Søg efter brugeren i databasen ud fra mailen.
2. Tjek om brugeren allerede er verificeret (is_verified). Hvis ja, omdirigeres du til login-siden.
3. Tjek om der findes et gyldigt token (verification_token) for brugeren. Hvis ikke, omdirigeres du til login-siden.
4. Tjek om tokenet er udløbet ved at sammenligne token_expires_at med den aktuelle tid. Hvis tokenet er udløbet, får brugeren mulighed for at anmode om et ny verifikationslink.
5. Hvis tokenet er gyldigt og ikke udløbet, opdateres brugerens status til verificeret (is_verified = 1), og tokenet fjernes fra databasen

Brugeren har 1 time til at verificere deres email. Hvis du ikke gør det indenfor denne tidsramme, skal du anmode om et nyt link. Hvis de for gennem sent et nyt link, for de også en ny token og et nyt expiry date

Nedenfor ser du hvordan verifikationen bliver håndteret i gennem de trin for oven

```

$user_email = _get_validate_user_email();

    $sql = "SELECT verification_token, token_expires_at,
is_verified FROM `users` WHERE user_email = :email";
    $stmt = $_db->prepare( $sql );

    $stmt->bindValue(":email", $user_email);
    $stmt->execute();

    $user = $stmt->fetch();

    if(!$user){
        throw new Exception("no user found", 401);
    }
    // TODO:check token after sql selector?
    if($user["verification_token"] === NULL){
        throw new Exception("no email found to be
verified", 400);
    }

    if($user && strtotime($user['token_expires_at']) >
time()){

        $sql = "UPDATE users SET verification_token=
NULL, token_expires_at= NULL, is_verified = 1
        WHERE verification_token = :token";
        $stmt = $_db->prepare($sql);

        $stmt->
>bindValue(":token",$user['verification_token']);
        $stmt->execute();

        $_SESSION['flash_state'] = "success";
        $_SESSION['flash_message'] = "Welcome to
boligsiden you can now login you your account";
        header('Location: /login');
        exit;

    } else if ($user &&
strtotime($user['token_expires_at']) < time()) {

```

Verificeringsprocessen følger principperne om defensiv programmering som beskrevet i Andrew Hoffmans *Web Application Security* aldrig stol på at en bruger er den, de udgiver sig for at være, før det er verificeret.

Verificeringslinket er tidsbegrænset til 1 time via et `token_expires_at` timestamp i databasen. Dette er et eksempel på time-based security tokens der ikke udløber er en sikkerhedsrisiko, da et stjålet eller lækket token vil forblive gyldigt på ubestemt tid. Ved at sætte en udløbstid minimeres vinduet for misbrug.

Processen overholder desuden GDPR-kravene om brugerverifikation ved at kræve bekræftelse af emailadressen sikres det at persondata kun knyttes til en reel og verificerbar bruger. Uverificerede brugere kan ikke logge ind og deres data eksponeres ikke i systemet.

3. trussel model

For systematisk at identificere og adressere sikkerhedsrisici i boligportalen har jeg anvendt STRIDE-modellen, udviklet af Microsoft. STRIDE er en struktureret metode til at kategorisere trusler mod et system ud fra seks angrebstyper: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service og Elevation of Privilege.

Formålet med trusselsmodellen er ikke blot at liste kendte angreb, men at tænke som en angriber, hvad kan gå galt, hvem vil angribe, og hvordan. "the main goal should be to build a map of the application that can be used later when prioritizing and deploying attack payloads". Trussel Modellen er derfor udarbejdet med udgangspunkt i boligportalens angreb overflade: login-endpoint, API-endpoints, filupload, admin-funktionalitet og session-håndtering.

Angrebsoverflade:

- Login og registrering (api-login.php, api-sign-up.php)
- Admin-funktionalitet (page-admin.php, api-add-new-property.php, api-delete-item.php)
- Filupload (api-upload.php)
- Bolig-interaktion (api-save-property.php, api-unsave-property.php)
- Kortvisning og søgning (api-search.php, api-get-map-item.php)
- Session og cookies

Trussel	Angrebsscenario	Implementeret modforanstaltning	Afsnit
Spoofing	Angriber logger ind som anden bruger via gættet password	<code>password_verify()</code> + rate limiting efter 5 forsøg	4.5
Spoofing	Session hijacking via stjålet session-cookie	<code>HttpOnly</code> , <code>Secure</code> , <code>SameSite=Lax</code>	4.7
Spoofing	Uverificeret bruger forsøger at logge ind	Email-verificering med tidsbegrænset token	2.3

Tampering	SQL injection via inputfelter	Prepared statements med named placeholders	4.1
Tampering	Command injection via filupload eller inputfelter	Ingen shell-funktioner brugt PHP-native filhåndtering	4.2
Tampering	XSS injektion af ondsindet JavaScript	<code>htmlspecialchars()</code> på alt output + CSP-header	4.3
Tampering	CSRF uautoriserede handlinger på vegne af bruger	CSRF-tokens + <code>hash_equals()</code> på alle POST-endpoints	4.4
Tampering	IDOR manipulation af <code>item_pk</code> for at påvirke andre brugeres data	<code>user_fk</code> bindes altid til session server-side	4.6
Tampering	Ondsindet filupload (PHP-fil forklædt som billede)	Whitelist-validering + <code>getimagesize()</code> + nyt tilfældigt filnavn	4.8
Tampering	Klient-side manipulation af formularfelter	Al validering sker server-side i <code>_validate_field()</code>	4.10
Repudiation	Ingen spor af sikkerhedshændelser	<code>error_log()</code> med <code>SECURITY:-</code> præfiks på alle sikkerhedshændelser	6
Repudiation	Angriber nægter at have forsøgt uautoriseret adgang	IP-logning ved fejlede loginforsøg og admin-adgangsforsøg	6
Information Disclosure	PHP-version afsløres i HTTP-headers	<code>header_remove("X-Powered-By")</code> + <code>server_tokens off</code> i Nginx	8
Information Disclosure	Stack trace eller databasefejl eksponeres til bruger	Generiske flash-beskeder tekniske fejl logges kun server-side	6
Information Disclosure	Credentials i kildekode lækker via Git	<code>.env</code> + <code>.gitignore</code> Kerckhoffs princip	8

Information Disclosure	Clickjacking siden indlejres i ondsindet iframe	<code>X-Frame-Options: DENY</code>	4.11
Information Disclosure	MIME-sniffing af uploadede filer	<code>X-Content-Type-Options: nosniff</code>	4.11
Denial of Service	Brute force mod login-endpoint	Rate limiting HTTP 429 efter 5 forsøg inden for 15 minutter	4.5
Denial of Service	Overbelastning via store filupload-requests	<code>upload_max_filesize 12M</code> <code>+ post_max_size 12M i</code> <code>.htaccess</code>	8
Elevation of Privilege	Almindelig bruger forsøger at tilgå <code>/admin</code>	RBAC-tjek <code>in_array("admin",</code> <code>\$_SESSION["user_role"])</code>	4.6
Elevation of Privilege	Uautoriseret bruger forsøger at kalde admin-API'er direkte	Session-validering + rolletjek i alle admin-endpoints	4.6

Tabellen viser at alle identificerede trusler er adresseret med konkrete modforanstaltninger. De primære åbne issues er session fixation (`session_regenerate_id()` ikke implementeret) og at rate limiting er session-baseret frem for database-baseret, hvilket gør den sårbar over for sessionsletning. Disse er dokumenteret som accepterede risici i det nuværende proof of concept.

4. Sikkerhedsanalyse

Som en generel tilgang til sikkerhed i boligportalen har jeg fulgt princippet om kun at tillade det, jeg eksplicit forventer aldrig mere. Dette afspejles i min `.htaccess`-konfiguration, hvor jeg med `RewriteRule` definerer præcis hvilke sider og API-endpoints der er tilgængelige. Alt andet afvises af Apache. På denne måde reduceres angrebsoverfladen markant, da en angriber ikke kan tilgå tilfældige PHP-filer direkte.

```
RewriteRule ^page-profile$ /pages/page-profile.php [NC,L]
RewriteRule ^$ /pages/page-index.php [NC,L]
RewriteRule ^page-map$ /pages/page-map.php [NC,L]
RewriteRule ^login$ /pages/page-login.php [NC,L]
```

4.1. SQL-injektion

SQL-injektion er et angreb, hvor en angriber forsøger at manipulere en SQL-forespørgsel ved at injicere ondsindet kode i et input felt for eksempel ved at skrive ' OR '1'='1 i et loginformular for at omgå password tjekket.

I alle mine SQL-forespørgsler bruger jeg prepared statements med named placeholders. Idéen er at sende SQL-strukturen til databasen adskilt fra dataen. Databasen modtager først skabelonen, og derefter sendes brugeren inputtet som en separat parameter. Database engine behandler altid inputtet som en ren værdi aldrig som en del af SQL-kommandoen. Ondsindet input som ' OR '1'='1 behandles derfor blot som en literal tekststreng og kan ikke ændre forespørgsels logik.

```
$sql = "call get_user_data_by_email(:email)";

$stmt = $_db->prepare($sql);
$stmt->bindValue(":email", $user_email);
$stmt->execute();

$user = $stmt->fetch();
```

Derudover validerer jeg alle input felter inden de når databasen, for eksempel kontrollerer `_validate_user_email()` i `_.php` at emailen overholder format, min- og maxlængde via PHPs `filter_var()` med `FILTER_VALIDATE_EMAIL`.

```
define("USER_EMAIL_MIN", 6);
define("USER_EMAIL_MAX", 50);
/**
 * @return false|string
 */
function _validate_user_email(): string|false {
    $user_email = filter_var(trim($_POST["user_email"] ?? ""), FILTER_VALIDATE_EMAIL);
    if (strlen($user_email) < USER_EMAIL_MIN) {
        throw new Exception("Email must be at least " . USER_EMAIL_MIN . " characters long", 400);
    }
    if (strlen($user_email) > USER_EMAIL_MAX) {
        throw new Exception("Email must be max " . USER_EMAIL_MAX . " characters long", 400);
    }
    if (!filter_var($user_email, FILTER_VALIDATE_EMAIL)) {
        throw new Exception("Invalid email $user_email", 400);
    }
    return $user_email;
}
```

4.2 Cross-Site Scripting (XSS)

XSS er et angreb, hvor en angriber forsøger at injicere ondsindet JavaScript i en webapplikation, som derefter afvikles i andre brugeres browsere. Som Andrew Hoffman beskriver det i *Web Application Security*: angrebet udnytter dårlig filtrering af output, og kan anvendes på tværs af mange typer applikationer.

Min grundregel er: stol aldrig på input fra klienten. Hverken GET-parametre, POST-indhold eller cookie-værdier. Og vigtigst: escape altid output.

Til dette bruger jeg en hjælpefunktion `_()` defineret i `_.php`, som kalder `htmlspecialchars()` på alle output. Det samme gælder `out()` i `router.php`. På den måde bliver tegn som `<`, `>` og `"` konverteret til harmløse HTML-entiteter, inden de sendes til browseren.

```
function _($text): void{  
    echo htmlspecialchars($text);  
}
```

Eksempel på brug i en skabelon:

```
<label for="username">Username must be between  
    <?= _(USER_USERNAME_MIN) ?> and  
    <?= _(USER_USERNAME_MAX) ?> characters.  
    <input min="<?= _(USER_USERNAME_MIN) ?>" max="<?= _(USER_USERNAME_MAX) ?>"  
</label>
```

Derudover har jeg i `.htaccess` sat headeren `X-XSS-Protection: 1; mode=block`, som beder browseren om at blokere sider, hvis den detekterer et XSS-forsøg.

Jeg har tilføjet en Content Security Policy (CSP) header i `.htaccess`, som begrænser hvilke ressourcer browseren må indlæse. `default-src 'self'` sikrer at alt blokeres som udgangspunkt, og kun eksplicit tilladte kilder som `unpkg.com` (Leaflet) og `kit.fontawesome.com` er whitelistede.

```
<IfModule mod_headers.c>  
    Header set X-Frame-Options "DENY"  
    Header set X-XSS-Protection "1; mode=block"  
    Header set X-Content-Type-Options "nosniff"  
    Header set Content-Security-Policy "default-src 'self'; script-src 'self' 'unsafe-inline' https://unpkg.com  
    https://kit.fontawesome.com; style-src 'self' 'unsafe-inline' https://unpkg.com; img-src 'self' data:  
    https://cdn-icons-png.flaticon.com https://*.basemaps.cartocdn.com https://placeholder.co; font-src 'self'  
    https://ka-f.fontawesome.com; connect-src 'self'"  
</IfModule>
```

CSP'en indeholder `'unsafe-inline'` under `script-src`, da kortsidens Leaflet-logik kræver adgang til PHP-genereret data direkte i HTML'en.

```

let items = <?= $items ?? []?>

items.forEach(item => {
  var marker = L.marker([item.lat, item.lon], {
    icon: L.divIcon({
      className: '',
      html: `
        <button
          class="marker ${item.type.replaceAll(" ", "_")}"
          data-item-pk="${item.pk}"
          onclick="showInfo(this)">
        </button>
      `
    })
  }, {
    item_pk: item.pk
  });
  markers.addLayer(marker)
});
map.addLayer(markers)

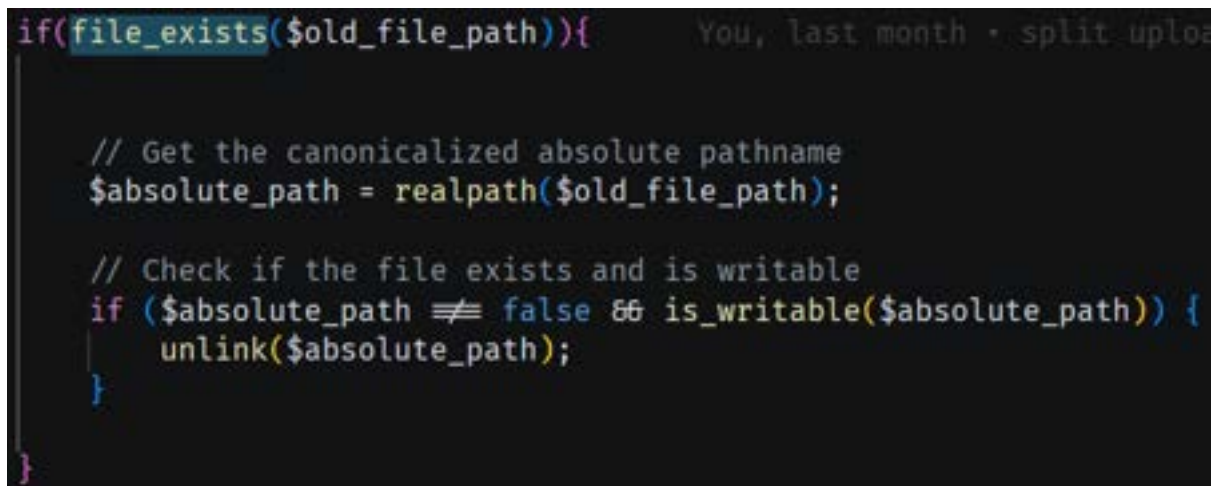
```

Dette svækker CSP'ens beskyttelse mod XSS en smule, da inline scripts tillades. En forbedring ville være at eksponere PHP-data via `window.mapItems` og flytte al script-logik til en ekstern fil, hvilket ville muliggøre fjernelse af `'unsafe-inline'` og styrke CSP'en yderligere. Denne afvejning er valgt bevidst, da den øvrige XSS-beskyttelse via `htmlspecialchars()` på alt output reducerer risikoen for at en angriber overhovedet kan injicere inline scripts

4.2 Command Injection

Command injection er et angreb hvor en angriber forsøger at injicere og afvikle shell-kommandoer på serveren ved at sende ondsindet input, for eksempel `; rm -rf /` i et input felt der bruges i et systemkald.

I min applikation er dette angreb vektor elimineret ved design. Jeg anvender ingen af PHPs funktioner der afvikler shell-kommandoer: `shell_exec()`, `exec()`, `system()` eller `passthru()`. Al filhåndtering sker udelukkende via PHPs egne filesystem-funktioner som `move_uploaded_file()` og `file_exists()`, der ikke fortolker shell-kommandoer.



```
if(file_exists($old_file_path)){           You, last month • split uploa

    // Get the canonicalized absolute pathname
    $absolute_path = realpath($old_file_path);

    // Check if the file exists and is writable
    if ($absolute_path !== false && is_writable($absolute_path)) {
        unlink($absolute_path);
    }
}
```

Det er et bevidst valg at holde applikationen fri af shell-kald, enhver funktionalitet der kunne kræve det er i stedet løst med PHP-native funktioner, som er sandboxede og ikke kan afvikle systemkommandoer direkte.

4.3 Cross-Site Request Forgery (CSRF)

Cross-site request forgery (også kendt som CSRF) er en websikkerhed sårbarhed, der gør det muligt for en angriber at få brugere til at udføre handlinger, som de ikke har til hensigt at udføre. Det giver en angriber mulighed for delvist at omgå den samme oprindelse politik, som er designet til at forhindre forskellige websteder i at forstyrre hinanden.

Jeg beskytter mod CSRF på to måder. For det første bruger jeg et CSRF-token i alle formularer der ændrer state, implementeret i `_.php` via `set_csrf()` og `is_csrf_token_valid()`. Token genererer en `bin2hex(random_bytes(50))`, som producerer en 100 tegn lang kryptografisk tilfældig hex-streng der er umulig at gætte, og gemmes i sessionen. Ved hvert POST-request sammenlignes token fra formularen med det gemte token i sessionen via `hash_equals()`, som beskytter mod timing attacks hvor en angriber måler responstiden for at gætte tokenet tegn for tegn.


```

function set_csrf()
{
    if (session_status() === PHP_SESSION_NONE) {
        session_start();
    }
    if (!isset($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(50));
    }
    echo '<input type="hidden" name="csrf_token" value="' . htmlspecialchars($_SESSION['csrf_token']) . '">';
}

function is_csrf_valid()
{
    if (session_status() === PHP_SESSION_NONE) {
        session_start();
    }
    if (!isset($_SESSION['csrf_token']) || !isset($_POST['csrf_token'])) {
        return false;
    }
    if (!hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'])) {
        return false;
    }
    if ($_SESSION['csrf_token'] !== $_POST['csrf_token']) {
        return false;
    }
    return true;
}

```

set_csrf() er implementeret i alle formularer der ændrer data, og is_csrf_token_valid() tjekkes i de tilsvarende API-endpoints:

- page-login.php → api-login.php
- page-sign-up.php → api-sign-up.php
- page-admin.php → api-upload.php, api-add-new-property.php
- page-profile.php → api-upload.php
- Gem/fjern bolig → api-save-property.php, api-unsave-property.php
- send_offer_request_email.php

kode eksempel

```

api-add-new-property.php X
apis > api-add-new-property.php > $e
11
12
13 session_start();
14 require_once __DIR__ . '/../routes.php';
15 require_once __DIR__ . '/../_php';
16 require_once __DIR__ . '/../db.php';
17
18 if ($_SERVER['REQUEST_METHOD'] === "POST") {
19
20     try{
21
22
23         if (!is_csrf_valid()) {
24             throw new Exception("Invalid CSRF token", 403);
25         }
26

```

```

catch ( Exception $e){

    error_log("Error: " . $e->getMessage() . " (Code: " . $e->getCode() . ")");

    $_SESSION['flash_state'] = "error";
    $message = $e->getMessage();
    switch (true) {
        case str_contains($message, "Invalid CSRF token"):
            $_SESSION['flash_message'] = "Invalid CSRF token";
            header('Location: /admin');
            exit;
    }
}

```

```

<form action="api-upload" method="POST" enctype="multipart/form-data">
    Select image to upload:
    <?php set_csrf();?>
    <input type="file" name="fileToUpload" id="fileToUpload" onChange="
    <input type="submit" value="Upload Image" name="submit">
</form>

```

Endpoints der kun læser data, som api-search.php og api-get-map-item.php, kræver ikke CSRF-beskyttelse da de ikke ændrer noget i databasen eller sessionen.

For det andet sætter jeg cookie-parametre med session_set_cookie_params(), som giver tre sikkerhedsfordele: SameSite=Lax forhindrer cookies i at blive sendt med cross-site requests, HttpOnly gør cookies utilgængelige fra JavaScript, og Secure sikrer at cookies kun sendes over HTTPS.

```

session_set_cookie_params([
    'lifetime' => 0,
    'path' => '/',
    'domain' => '',
    'secure' => true, // Only send over HTTPS
    'httponly' => true,
    'samesite' => 'Lax' // or 'Strict'
]);

```

Same Site-cookies - SameSite er en browser sikkerhedsmekanisme, der bestemmer, hvornår et websteds cookies inkluderes i anmodninger, der stammer fra andre websteder.

Da anmodninger om at udføre følsomme handlinger typisk kræver en godkendt sessionscookie, kan de relevante SameSite-begrænsninger forhindre en angriber i at udløse disse handlinger på tværs af websteder. Siden 2021 har Chrome som standard håndhævet Lax SameSite-begrænsninger. Da dette er den foreslåede standard, forventer vi, at andre større browsere vil anvende denne adfærd i fremtiden.

SameSite=Lax: Sender cookies til anmodninger på samme websted og navigationer på topniveau (f.eks. klik på et link til webstedet). Den sender ikke cookies på underanmodninger

på tværs af websteder, som f.eks. billeder eller iframes. Dette er standarden i de fleste moderne browsere.

SameSite=Strict: Sender aldrig cookies på anmodninger på tværs af websteder. Cookies sendes kun, når anmodningen stammer fra præcis det samme websted, der har indstillet dem.

Dette er relevant for særligt følsomme handlinger som at oprette en ny bolig i databasen via admin-siden. Her kræves det både at brugeren er logget ind og har admin-rollen, en handling der aldrig bør kunne triggere fra et eksternt websted, uanset context.

```
session_set_cookie_params([
    'lifetime' => 0,
    'path' => '/',
    'domain' => '',
    'secure' => true,
    'httponly' => true,
    'samesite' => 'Strict'
]);
```

4.4 Brudt autentifikation og brute force

En bruger autentificeres ved at sammenligne det indtastede password med det hashed password i databasen via PHPs `password_verify()`. Passwordet gemmes aldrig i klartekst hverken i databasen eller i sessionen. Adgangskoden fjernes eksplicit fra sessionen efter login.

Jeg har implementeret rate limiting via en session-baseret tæller der registrerer fejlede loginforsøg per IP-adresse. Efter 5 fejlede forsøg inden for 15 minutter blokeres yderligere forsøg og serveren returnerer HTTP 429:

```

$ip = $_SERVER['HTTP_X_REAL_IP'] ?? $_SERVER['REMOTE_ADDR'];
$max_attempts = 5;
$lockout_time = 15 * 60; // 15 minutter i sekunder

// Initialiser tæller
if (!isset($_SESSION['login_attempts'][$ip])) {
    $_SESSION['login_attempts'][$ip] = ['count' => 0, 'last_attempt' => time()];
}

$attempts = $_SESSION['login_attempts'][$ip];

// Reset hvis lockout-tid er udløbet
if (time() - $attempts['last_attempt'] > $lockout_time) {
    $attempts['count'] = 0;
}

// Bloker hvis for mange forsøg
if ($attempts['count'] >= $max_attempts) {
    throw new Exception("SECURITY: Brute force attempt from IP: $ip", 429); // Too Many Requests
}

```

Sikkerhedshændelser logges med **SECURITY**:-præfiks som beskrevet i afsnit 6.

En begrænsning ved denne implementering er at den session-baserede tæller nulstilles hvis sessionen slettes eller serveren genstartes. En mere robust løsning ville gemme forsøgs tælleren i databasen, så den overlever på tværs af sessioner og genstarter

4.5 Brudt adgangskontrol (IDOR)

Insecure Direct Object Reference (IDOR) opstår, når en bruger kan tilgå ressourcer, de ikke har rettighed til, blot ved at gætte eller ændre et ID i en forespørgsel.

Jeg beskytter mod dette på to måder. For det første validerer `_validate_pk()` i `_.php` at alle primærnøgler er præcis 50 tegn lange og gyldige hexadecimale strenge da jeg genererer nøgler med `bin2hex(random_bytes(25))`, er de umulige at gætte.

```

function _validate_pk(string $pk): string {
    // Check length (must be exactly 50 characters)
    if (strlen($pk) !== 50) {
        throw new Exception("Primary key must be exactly 50 characters long", 400);
    }

    // Check if the pk is a valid hexadecimal string
    if (!ctype_xdigit($pk)) {
        throw new Exception("Primary key must be a valid hexadecimal string", 400);
    }

    return $pk;
}

```

For det andet tjekker admin-endpoints i `api-add-new-property.php` aktivt om den indloggede bruger har admin-rollen, inden en handling udføres:

```

if (!is_csrf_valid()) {
    throw new Exception("Invalid CSRF token", 403);
}

// check if the user is actually admin before adding property
if (!isset($_SESSION["user"])) {
    throw new Exception("no user found", 401);
}

if(!in_array("admin", $_SESSION["user"]["user_role"])){
    throw new Exception("user not admin", 403);
}

```

4.6 Session Hijacking og Session Fixation

Session hijacking er et angreb, hvor en angriber forsøger at stjæle en brugers session-ID for at udgive sig for at være dem. Når en bruger logger ind, opretter PHP en session og sender et session-ID til browseren via en cookie. Hvis en angriber kan få fat i dette ID for eksempel via XSS eller netværkssniffing kan de sende det samme ID i deres egne requests og serveren vil behandle dem som den legitime bruger.

Session fixation er et relateret men anderledes angreb. Her planter angriberen et session-ID *inden* login:

1. Angriberen besøger login-siden og får et session-ID — f.eks. **ABC123**
2. Angriberen sender et link til offeret med det kendte ID:
<http://boligportal.dk/login?PHPSESSID=ABC123>
3. Offeret klikker på linket og logger ind med ID **ABC123**
4. Nu kender angriberen session-ID'et for en autentificeret bruger og kan udgive sig for at være dem

Jeg beskytter mod begge angreb med tre foranstaltninger:

session_regenerate_id(true) kaldes efter et succesfuldt login, inden brugerdata gemmes i sessionen. Den udsteder et nyt session-ID og sletter det gamle — selv hvis angriberen kendte **ABC123** inden login, er det ID, nu slettet og erstattet med et nyt ID som kun offeret kender. **true**-parameteren sikrer at den gamle session-fil slettes på serveren.

```

session_regenerate_id(true);

// put user in session
$_SESSION['user'] = [
    'user_pk'      => $user['user_pk'],
    'user_email'   => $user['user_email'],
    'user_username' => $user['user_username'],
    'user_lastname' => $user['user_lastname'],
    'user_forename' => $user['user_forename'],
    'user_image'   => $user['user_img'],
    'user_role'    => explode(',', $user_role['role_list']),
];

```

HttpOnly forhindrer at JavaScript kan læse session-cookien — selv hvis en angriber injicerer ondsindet JavaScript via XSS, kan scriptet ikke tilgå `document.cookie` og dermed ikke stjæle session-ID'et.

Secure sikrer at session-cookien aldrig sendes over en ukrypteret HTTP-forbindelse, så den ikke kan opsnapes af en angriber der lytter på netværket.

```

session_set_cookie_params([
    'lifetime' => 0,
    'path' => '/',
    'domain' => '', // Adjust as needed
    'secure' => true, // Only send over HTTPS
    'httponly' => true,
    'samesite' => 'Strict' // or 'Strict'
]);

```

`lifetime => 0` betyder at cookien slettes når browseren lukkes, så et gammelt session-ID ikke overlever til næste session.

Angreb	Modforanstaltning
Session hijacking via XSS	HttpOnly , JS kan ikke læse cookien
Session hijacking via netværk	Secure , cookie sendes kun over HTTPS

Session fixation	<code>session_regenerate_id(true)</code> , nyt ID ved login
------------------	---

4.7 Usikker filoverførsel

Filupload er en klassisk angrebsvektor, en angriber kan forsøge at uploade en PHP-fil forklædt som et billede for at afvikle kode på serveren.

Jeg håndterer dette i `_validate_uploaded_file()` i `_.php` med tre kontroller: først verificerer `getimagesize()` at filen faktisk er et billede på billeddata-niveau, dernæst tjekkes filstørrelsen (maks 500 KB), og til sidst kontrolleres filendelsen mod en whitelist af tilladte typer.

```
// =====
define("MAX_FILE_SIZE", 500000);
define("ALLOWED_TYPES", ["jpg", "png", "jpeg", "webp"]);
function _validate_uploaded_file(array $file): array{

    $check = getimagesize($file["tmp_name"]);
    if($check == false) {
        return ['valid' => false, 'message' => "sorry your file is not a valide image"];
    }

    if ($file["size"] > MAX_FILE_SIZE) {
        return ['valid' => false, 'message' => "sorry your image is too large"];
    }

    $imageFileType = strtolower(pathinfo($file["name"], PATHINFO_EXTENSION));
    if (!in_array($imageFileType, ALLOWED_TYPES)){
        return ['valid' => false, 'message' => "sorry only JPG JPEG PNG and WEBP files allowed"];
    }

    return ['valid' => true, 'message' => ""];
}
```

Derudover omdøbes alle uploadede filer til et nyt tilfældigt navn med `bin2hex(random_bytes(25))`, så den originale filnavn og extension fra klienten aldrig bruges direkte.

4.8 XML External Entity (XXE) og serialisering

XXE-angreb udnytter XML-parsere til at læse filer på serveren eller kontakte interne systemer. Min applikation anvender ikke XML-parsing al kommunikation mellem klient og server sker via HTML-formularer og JSON. JSON er at foretrækket over XML i API-kommunikation, da JSON-parsere ikke er sårbare over for XXE på samme måde.

Da jeg ikke bruger `simplexml_load_string()` eller lignende XML-funktioner, er denne angrebsvektor ikke relevant for mit projekt.

4.9 Klient-side manipulation

En angriber kan manipulere data der sendes fra browseren, for eksempel ændre skjulte formularfelter, omgå JavaScript-validering eller sende egne HTTP-requests med Burp Suite.

Min tilgang er at al validering sker på server-side i PHP, uanset hvad der sker på klienten. Jeg har implementeret en central valideringsfunktion `_validate_field()` i `_.php`, der bruges til at validere alle felter i bolig-formularen ud fra et sæt regler defineret i `validation_rules.php`. Den tjekker type, min/max-værdier og regex-mønstre for hvert felt.

```
// validate and saniatice all the data from the form
$validated_data = [];
foreach ($property_rules as $field => $rules) {
    $value = $_POST[$field] ?? null;
    $validated_data[$field] = _validate_field($field, $value, $rules);
};
```

4.10 Clickjacking og MIME-sniffing

Clickjacking er et angreb, hvor en angriber indlejrer min side i en usynlig `<iframe>` på en ondsindet side og narrer brugere til at klikke på elementer de ikke kan se. MIME-sniffing er når en browser forsøger at gætte en fils indholdstype, hvilket kan udnyttes til at afvikle en uploadet fil som JavaScript.

Begge angreb imødegås via headers i `.htaccess`:

```
Header set X-Frame-Options "DENY"
Header set X-XSS-Protection "1; mode=block"
Header set X-Content-Type-Options "nosniff"
```

X-Frame-Options: DENY forhindrer fuldstændigt at siden kan indlejres i en iframe. **X-Content-Type-Options: nosniff** tvinger browseren til at respektere den angivne Content-Type og ikke gætte filtypen selv.

4.11 Supply chain-angreb

Supply chain-angreb retter sig mod de eksterne afhængigheder, en applikation bruger. Kompromitteres en pakke, kompromitteres alle der bruger den.

Jeg bruger Composer til PHP-afhængighed (primært PHPMailer). Ved at committe `composer.lock` til Git sikrer jeg at alle miljøer bruger præcis de samme versionsspecifikke pakker, ikke blot de nyeste, som kunne være kompromitterede.

ikke blot de nyeste, som kunne være kompromitterede. Kendte sårbarheder i Composer-pakker kan verificeres med `composer audit`

4.12 CORS-miskonfiguration

Cross-Origin Resource Sharing (CORS) styrer, hvilke domæner der må kalde ens API. En fejlkonfiguration kan tillade ondsindede sider at kalde mine API-endpoints på vegne af en indlogget bruger.

Da mine API-endpoints er server-rendered og ikke et åbent REST API beregnet til at tilgås fra andre domæner, har jeg ikke sat eksplicitte CORS-headers. Apache tillader som standard ikke cross-origin requests til PHP-endpoints, hvilket er det sikre udgangspunkt. Alle API-kald forventes at komme fra samme oprindelse.

5. Sikkerhedslogging

Jeg håndterer fejl og sikkerhedshændelser i alle mine API-endpoints via en central try/catch-blok med et switch-case mønster. Denne tilgang adskiller bevidst hvad der logges server-side fra hvad brugeren ser. Alle fejl logges med `error_log()` som skriver til serverens log aldrig til browseren:

```
if(!in_array("admin", $_SESSION["user"]["user_role"])){
    throw new Exception("user not admin", 403);
}

catch ( Exception $e){
    error_log("Error: " . $e->getMessage() . " (Code: " . $e->getCode() . ")");
}
```

Brugeren får i stedet en generisk flash-besked via session:

```
case str_contains($message, "user not admin"):
    $_SESSION['flash_message'] = "You do not have permission to access this page";
    header('Location: /admin');
    exit;
```

Flash-beskeden vises som en farvet pop-up, rød ved fejl og indeholder aldrig tekniske detaljer som stack traces, databasestruktur eller framework-versioner. Dette er et bevidst valg for at forhindre information leakage, som hackere bruger til rekognoscering.

Specifikke fejlskeder kan afsløre backend-frameworks, system versioner og database struktur, hvilket gør det markant nemmere at udføre angreb som SQL-injektion. Switch-case mønsteret bruges til at skelne mellem kendte fejlscenarier og uventede fejl:


```

catch(Exception $e){
    // Rollback any open transactions.
    if ($_db->inTransaction()) {
        $_db->rollBack();
    }

    error_log("Error: " . $e->getMessage() . " (Code: " . $e->getCode() . ")");

    // Set HTTP status code (default to 500 if not specified)
    http_response_code($e->getCode() ≥ 400 ? $e->getCode() : 500);
    $_SESSION['flash_state'] = "error";

    // Handle specific errors with switch
    $message = $e->getMessage();
    switch (true) {
        case str_contains($message, "user_email") && str_contains($message, "Duplicate entry"):
            $_SESSION['flash_message'] = "Email already exists";
            header('Location: /sign-up');
            exit;

        case str_contains($message, "Invalid CSRF token"):
            $_SESSION['flash_message'] = "Invalid CSRF token";
            header('Location: /sign-up');
            exit;
    }
}

```

Min tilgang til sikkerhedslogging er bygget på AAA-modellen, Authentication, Authorization of Accounting. Accounting er det tredje ben i modellen og handler om at logge hvad der sker i systemet, hvem der gjorde det, hvornår og fra hvilken IP.

Problemerne ligger ofte i accounting, man skriver for få beskeder til loggen og dermed misser, hvis et sikkerhedsbrud sker, eller en fejl opstår. Det er præcis dette jeg adresserer ved at logge sikkerhedshændelser via `error_log()` i alle mine API-endpoints. Jeg skelner bevidst mellem hvad der logges server-side og hvad brugeren ser. Alle fejl logges til serverens log via Docker:

```

catch ( Exception $e){

    error_log("Error: " . $e->getMessage() . " (Code: " . $e->getCode() . ")");
}

```

Dette sikrer at CSRF-fejl returnerer 403, valideringsfejl returnerer 400, og uventede fejl returnerer 500 i stedet for altid at returnere 200 OK, som ville skjule fejl for monitoring-værktøjer.

Noget som er godt at tænke over til produktionen er at loggen gemmes kun i Docker-containerens stdout og persisteres ikke på tværs af genstarter. I produktion ville man sende logs til et persistent system som f.eks. en database eller en dedikeret logtjeneste.

En yderligere forbedring ville være automatisk notifikation til administrator hvis samme IP gentagne gange forsøger uautoriseret adgang til admin-siden, dette ville give aktiv overvågning frem for passiv logging. OWASP anbefaler at logge hvem, hvad, hvornår og fra hvilken IP for sikkerhedskritiske hændelser

6. Transport Layer Security (TLS)

"TLS is a security protocol designed to facilitate privacy and data security for communication over the internet". TLS garanterer tre ting for kommunikationen mellem klient og server: kryptering (data skjules fra tredjepart), integritet (data kan ikke manipuleres undervejs), og autentificering (serveren er den den udgiver sig for at være).

Uden TLS sendes al kommunikation i klartekst over HTTP herunder passwords, session-cookies og persondata. En angriber på samme netværk kunne med et man-in-the-middle-angreb læse eller ændre trafikken. Dette er præcis grunden til at mine session-cookies er sat med Secure-attributten

Secure-attributten betyder at cookien ikke fysisk kan sendes over en ukrypteret HTTP-forbindelse men den forudsætter at serveren faktisk kører HTTPS.

Overvejelser for produktion: I mit nuværende setup kører applikationen lokalt via Docker på `http://127.0.0.1:8000` uden TLS-certifikat. I produktion ville jeg implementere TLS via Let's Encrypt og Certbot, som jeg har gjort i et andet projekt med DNS-validering. Processen er:

- Server med et domæne peget på via DNS
- Certbot udsteder et gratis TLS-certifikat fra Let's Encrypt
- Certbot konfigurerer automatisk Apache til at redirecte HTTP → HTTPS
- Certifikatet fornyes automatisk hver 90. dag

7. Reverse Proxy Nginx

Som et ekstra sikkerhedslag foran PHP-serveren har jeg tilføjet Nginx som reverse proxy. En reverse proxy modtager alle indkommende requests fra brugeren og videresender dem til PHP-containeren internt, brugeren kommunikerer aldrig direkte med PHP-serveren.

Dette giver flere sikkerhedsfordele. Netværket er opdelt i to isolerede Docker-netværk: `webnet` der forbinder Nginx og PHP-containeren, og `dbnet` der forbinder PHP-containeren og MariaDB. Det betyder Nginx aldrig kan tale direkte med databasen, selv hvis Nginx blev kompromitteret, har den ingen adgang til databaselaget.

PHP-containeren har ingen eksponerede porte udadtil kun Nginx er tilgængeligt på port 8000. En angriber der scanner efter åbne porte ser kun Nginx, ikke PHP eller Apache bag ved.

I `nginx.conf` er `server_tokens off` sat for at forhindre at Nginx afslører sin version i HTTP-headers samme princip som `expose_php = Off` og `header_remove("X-Powered-By")` i PHP:

```

upstream app {
    server web:80;
}

server {
    listen 80;

    # Skjul nginx version fra headers
    server_tokens off;

    # Begræns request størrelse (beskytter mod store uploads der omgår PHP)
    client_max_body_size 12M;

    # Tilføj security headers her også
    add_header X-Frame-Options "DENY";
    add_header X-Content-Type-Options "nosniff";

    location / {
        proxy_pass http://app;

        # Send rigtig IP videre til PHP som bruges i login og admin page
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}

```

X-Real-IP-headeren er særligt vigtig for brute force-beskyttelsen i `api-login.php` uden den ville PHP modtage Nginx's interne IP i stedet for brugerens rigtige IP, og rate limiting ville tælle alle brugere som én:

```

$ip = $_SERVER['HTTP_X_REAL_IP'] ?? $_SERVER['REMOTE_ADDR'];
$max_attempts = 5;
$lockout_time = 15 * 60; // 15 minutter i sekunder

// Initialiser tæller
if (!isset($_SESSION['login_attempts'][$ip])) {
    $_SESSION['login_attempts'][$ip] = ['count' => 0, 'last_attempt' => time()];
}

```

8. Konfiguration og Serveropsætning

Som Bruce Schneier formulerer det: *"Security is a process, not a product"* (https://www.schneier.com/essays/archives/2000/04/the_process_of_secur.html). Sikker serverkonfiguration er ikke noget man gør én gang, det er løbende beslutninger om hvad der eksponeres og hvad der skjules.

.htaccess, whitelist af endpoints

Som bedste praksis følger jeg princippet om kun at tillade det, man eksplicit ønsker. I `.htaccess` definerer jeg præcis hvilke sider og API-endpoints der er tilgængelige via `RewriteRule`. Alt der ikke matcher en regel afvises af Apache, en angriber kan ikke tilgå tilfældige PHP-filer direkte

Derudover sætter `.htaccess` grænser for filupload så en angriber ikke kan overbelaste serveren med store filer:

```
php_value post_max_size 12M
php_value upload_max_filesize 12M
```

Sikkerhedsheadere

Tre sikkerheds headere er sat i `.htaccess`, og CSP-headeren sættes i PHP via `_.php` da den er for lang til én linje i Apache-konfigurationen

```
<IfModule mod_headers.c>
    Header set X-Frame-Options "DENY"
    Header set X-XSS-Protection "1; mode=block"
    Header set X-Content-Type-Options "nosniff"
</IfModule>
```

Miljøvariabler og Kerckhoffs princip

Kerckhoffs princip siger at et system skal være sikkert selvom alt ved systemet, undtagen nøglen, er offentligt vidend. Dette er direkte anvendt i min håndtering af credentials: kildekoden kan ligge åbent på github, men alle hemmeligheder (database-password, Gmail-app-password) er fjernet fra koden og placeret i en `.env`-fil:

```
MAIL_FROM=
MAIL_PASSWORD=
MAIL_FROM_NAME=
ADMIN_EMAIL=
```

`.env` er udelukket fra Git via `.gitignore`, mens `.env.example` committes som skabelon. Koden læser værdierne via `$_ENV`:

```
try{
    $dbConnection = 'mysql:host=' . $_ENV['DB_HOST'] . ';dbname=' . $_ENV['DB_NAME'] . ';charset=utf8mb4';
    // utf8 every character in the world
    // mb4 every character and also emojis
    $options = [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION, // try-catch
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC // ['nickname']
        //PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_OBJ // →nickname
        // PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_NUM // [[2],[],[ ]]
    ];
    $_db = new PDO($dbConnection, $_ENV['DB_USER'], $_ENV['DB_PASSWORD'], $options);
}
```

Docker og isolation

Applikationen kører i Docker med to adskilte services, **web** (Apache/PHP) og **Mariadb**, som kommunikerer via Dockers interne netværk. Databasen er dermed ikke direkte tilgængeligt fra internettet, kun fra webservicen

CI/CD og automatiseret sikkerhed

Som en naturlig forlængelse af **.env**-løsningen ville en GitLab CI/CD pipeline håndtere secrets via GitLab CI/CD Variables frem for manuelle **.env**-filer på serveren. Dette sikrer at credentials aldrig berører filesystemet direkte en angriber der kompromitterer én server får ikke adgang til credentials fra andre miljøer.

Derudover tilbyder GitLab SAST-scanning (Static Application Security Testing) som automatisk analyserer kildekoden for kendte sikkerheds sårbarheder ved hvert push. Dette er et konkret værktøj til løbende at efterleve Bruce Schneiers princip om at sikkerhed er en proces, ikke noget man sætter op én gang og glemmer.

php.ini-overvejelser

I produktion burde følgende PHP-indstillinger sættes for at reducere information lækage:

ini

display_errors = Off ; PHP-fejl vises ikke til brugeren

expose_php = Off ; Fjerner PHP-versionsinfo fra HTTP-headers

log_errors = On ; Fejl logges server-side i stedet

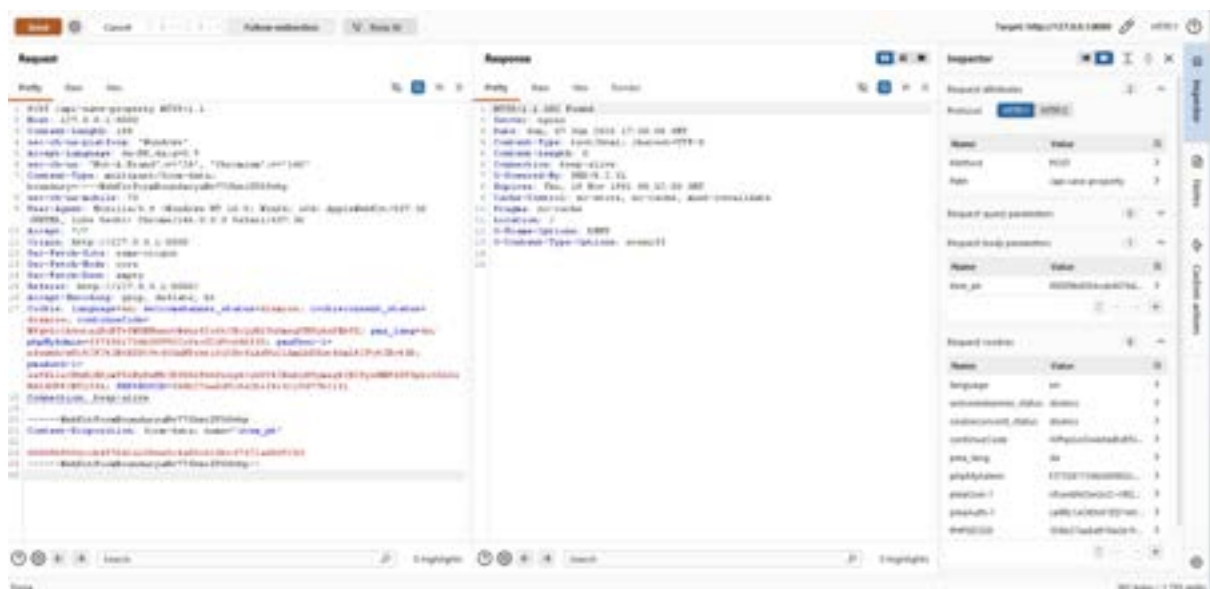
expose_php = Off er særligt relevant uden den afslører Apache-headeren **X-Powered-By: PHP/8.2** hvilken PHP-version der kører, hvilket gør det nemmere for en angriber at finde kendte CVE-sårbarheder for netop den version.

9. Angrebs Beviser og Penetrationstest

For at verificere at de implementerede sikkerhedsforanstaltninger fungerer i praksis, har jeg udført manuel penetrationstest ved hjælp af Burp Suite. Formålet var at simulere reelle angreb scenarier mod min egen applikation og dokumentere resultaterne.

9.1 CSRF manglende token

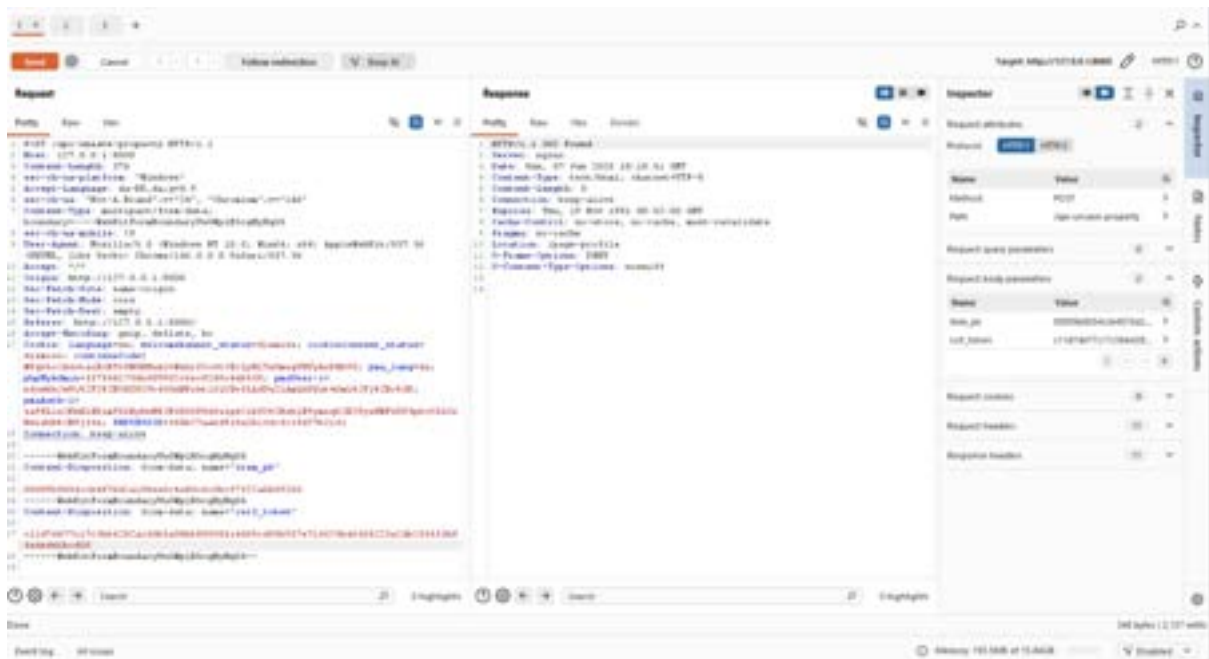
Det første testscenarie undersøgte hvad der sker når et POST-request sendes til **api-save-property** uden et CSRF-token. Via Burp Suite Repeater blev token-feltet fjernet fra request body og requestet sendt manuelt.



Resultatet var en **HTTP 302** redirect til / serveren afviste requestet korrekt fordi **is_csrf_token_valid()** returnerede **false** når **\$_POST['csrf_token']** ikke var sat.

9.2 CSRF manipuleret token

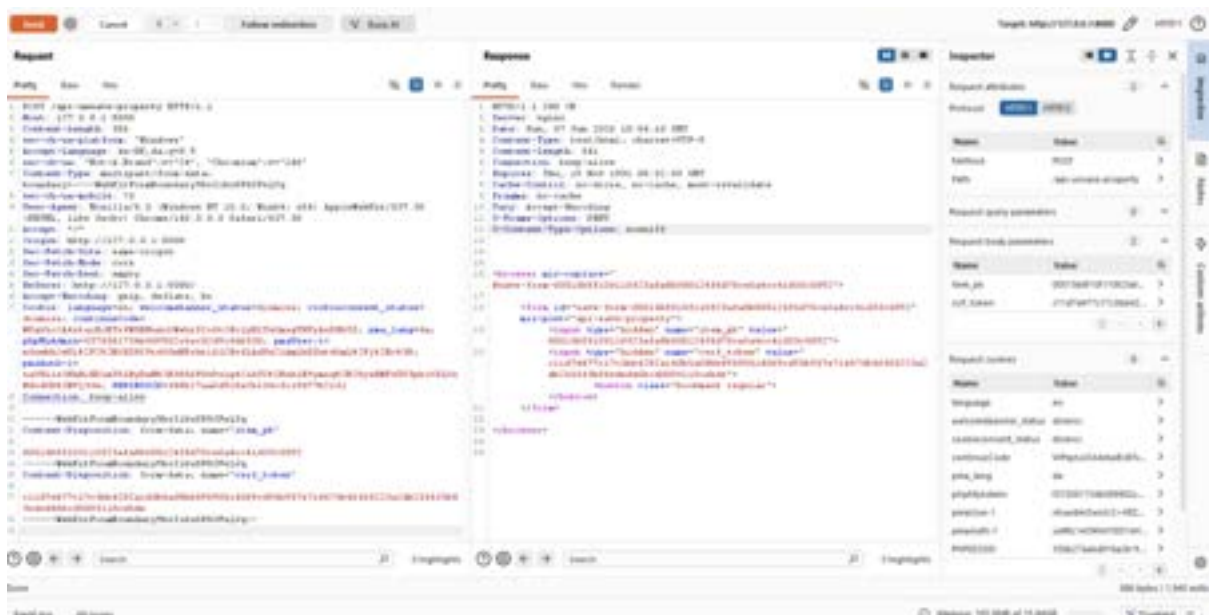
Det andet testscenarie undersøgte hvad der sker når et request sendes med et forkert CSRF-token i dette tilfælde et for kort token der ikke matcher det gemte session-token.



Serveren afviste også dette request med **HTTP 302**. `hash_equals()` sammenligner tokens i konstant tid og returnerer **false** når tokenerne ikke matcher uanset om tokenet mangler helt eller blot er forkert.

9.3 IDOR manipulation af item_pk

Det tredje testscenarie undersøgte, om en bruger kunne manipulere `item_pk` i et POST-request til `api-unsaved-property` for at påvirke en anden brugers gemte boliger.



Requestet returnerede **HTTP 200** men ingen data blev ændret i databasen. Dette skyldes at SQL-queryen altid binder brugerens session-PK server-side:


```
$sql = "DELETE FROM `saved_property` WHERE user_fk = :user_pk AND property_fk = :property_pk";
$stmt = $db->prepare($sql);
$stmt->bindValue(":property_pk",$item_pk );
$stmt->bindValue(":user_pk",$user_pk );
$stmt->execute();
```

`:user_pk` hentes udelukkende fra `$_SESSION["user"]["user_pk"]` og aldrig fra klienten. En angriber kan sende et vilkårligt `item_pk`, men DELETE-queryen finder ingen matchende række fordi `user_fk` ikke stemmer overens og sletter ingenting. IDOR-angrebet er dermed ineffektivt.

9.4 Statisk kodeanalyse Psalm

Udover manuel Burp Suite-testning blev Psalm kørt som statisk analysator på kodebasen. Psalm identificerede tre reelle fund der efterfølgende blev rettet:

- Forkert operator (`&&` i stedet for `||`) i admin-rolletjekket i `page-admin.php` kunne i teorien have tilladt uautoriseret adgang
- Typo `$_post` i stedet for `$_POST` i `session_utils.php` ville have forhindret korrekt læsning af POST-data
- Død kode efter `echo $e` i `send_offer_request_email.php` ville have eksponeret exception-detajler til brugeren

De resterende Psalm-fejl var falske positive relateret til projektets include-baserede arkitektur, som Psalm ikke kan analysere korrekt på tværs af filer.

```
-----
60 errors found
-----
```

```
185 other issues found.
```

```
You can display them with --show-info=true
```

```
-----
Psalm can automatically fix 38 of these issues.
```

```
Run Psalm again with
```

```
--alter --issues=UnusedVariable --dry-run
```

```
to see what it can fix.
```

```

ERROR: UnusedVariable - pages/page-profile.php:18:9 - $title is never referenced or the value is not used (see https://psalm.dev/077)
    $title = "Profile";

ERROR: UnusedVariable - pages/page-profile.php:29:13 - $viewed_homes is never referenced or the value is not used (see https://psalm.dev/077)
    $viewed_homes = get_viewed_homes();

ERROR: UnusedVariable - pages/page-profile.php:74:21 - $items is never referenced or the value is not used (see https://psalm.dev/077)
    $items = $viewed_homes;

ERROR: UnusedVariable - pages/page-profile.php:75:21 - $scroller_header is never referenced or the value is not used (see https://psalm.dev/077)
    $scroller_header = "Your recently viewed homes!";

ERROR: UnusedVariable - pages/page-sign-up.php:15:1 - $title is never referenced or the value is not used (see https://psalm.dev/077)
    $title = "Sign up";

```

10. Referencer

Bøger:

- Hoffman, A. (2024). *Web Application Security* (2nd ed.). O'Reilly Media. ISBN: 9781098143930

OWASP:

- OWASP Top 10: <https://owasp.org/www-project-top-ten/>
- OWASP Logging Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
- OWASP PHP Configuration Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/PHP_Configuration_Cheat_Sheet.html
- OWASP Security by Design Principles: https://cheatsheetseries.owasp.org/cheatsheets/Security_by_Design_Principles_Cheat_Sheet.html
- OWASP CSRF Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- OWASP Juice Shop: <https://pwning.owasp-juice.shop/companion-guide/latest/part1/categories.html>

Artikler og blogs:

- Schneier, B. (2000). *The Process of Security*. https://www.schneier.com/essays/archives/2000/04/the_process_of_security.html
- Cloudflare. *What is HTTPS?* <https://www.cloudflare.com/learning/ssl/what-is-https/>
- Let's Encrypt. *How it works*. <https://letsencrypt.org/how-it-works/>
- GitHub Blog. (2021). *Write more secure code with OWASP Top 10 Proactive Controls*. <https://github.blog/2021-12-06-write-more-secure-code-owasp-top-10-proactive-controls/>
- GitHub Blog. (2021). *Blue teaming: create a security advisory process*. <https://github.blog/2021-11-02-blue-teaming-create-security-advisory-process/>

- PortSwigger. *CSRF*. <https://portswigger.net/web-security/csrf>
- PortSwigger. *IDOR*. <https://portswigger.net/web-security/access-control/idor>
- Zend. *Tips for PHP Security*. <https://www.zend.com/blog/tips-php-security>

Værktøjer brugt:

- Burp Suite Community Edition <https://portswigger.net/burp>
- Psalm Static Analysis <https://psalm.dev>

11. Tidsestimat

Det samlede tidsforbrug på projektet estimeres til cirka 100 timer. Arbejdet består primært af implementering af sikkerhedsfunktioner, opsætning af Docker-miljøet, penetrationstest med Burp Suite og Psalm samt udarbejdelse af rapporten. De første uger fra 1. maj blev brugt på at bygge applikationen og implementere sikkerhedstiltag, mens den afsluttende periode fra 1. juni fokuserede på testning, dokumentation og rapportskrivning.

12. Konklusion og Refleksion

Dette projekt har jeg sat mig for mål om at lave et mere sikkerhedsmæssigt gennemtænkt boligportal som proof of concept, en applikation som en virksomhed kan migrere eksisterende funktionalitet over til med sikkerhed som et grundlæggende designprincip frem for en eftertanke.

Prevent → Detect → Recover-modellen kan projektets sikkerhedsniveau evalueres i tre lag:

Prevent forebyggelse af angreb er det lag jeg har mest fokus på. Alle kendte angrebsvektorer fra kurset er adresseret: SQL injection via prepared statements, XSS via `htmlspecialchars()` og CSP-headers, CSRF via kryptografiske tokens og SameSite-cookies, brute force via rate limiting, uautoriseret adgang via RBAC og PoLP, usikker filupload via whitelist-validering og omdøbning af filer, og informationslækage via generiske fejlbeskeder og `error_log()`.

Detect opdagelse af angreb er implementeret via sikkerhedslogging med SECURITY:- præfiks i `error_log()`, som gør det muligt at filtrere sikkerhedshændelser fra almindelige fejl. Korrekte HTTP-statuskoder (403, 429, 500) gør det muligt for monitoring-værktøjer at opdage unormale mønstre. Dette bygger på AAA-modellens Accounting-princip logging af hvem, hvad og hvornår for sikkerhedskritiske hændelser.

Recover begrænsning af skade er adresseret via transaktioner med `rollback()` i alle database-operationer, Docker-isolation der begrænser hvad en kompromitteret container kan tilgå, og miljøvariabler der sikrer at credentials ikke kompromitteres hvis kildekoden lækker.

Penetrationstest med Burp Suite og statisk analyse med Psalm verificerede at de implementerede sikkerhedsforanstaltninger fungerer i praksis og identificerede tre reelle kodefejl der efterfølgende blev rettet

Begrænsninger og åbne issues: Applikationen er ikke deployeret med TLS i dette projekt, selvom implementeringen er dokumenteret og udført i et andet projekt med Let's Encrypt og Certbot. 'unsafe-inline' i CSP-headeren svækker XSS-beskyttelsen en smule.

Session-regenerering efter login (`session_regenerate_id()`) er ikke implementeret, og phpMyAdmin samt MariaDB har eksponerede porte i udviklingsmiljøet der skal lukkes i produktion.

Refleksion

Det mest værdifulde læringsudbytte har ikke været de individuelle sikkerhedstiltag isoleret set, men forståelsen af at de fungerer i lag defense in depth. En enkelt foranstaltning er aldrig nok; det er kombinationen af prepared statements, output escaping, CSRF-tokens, sikkerhedsheadere, logging og korrekt serverkonfiguration der tilsammen hæver barren for en angriber. Som Bruce Schneier formulerer det: "Security is a process, not a product" og det er præcis den tankegang dette projekt har forsøgt at efterleve, fra den første linje kode til den afsluttende penetrationstest.